Xử Lý Tín Hiệu Số
HK261 – CO2036

BÀI THỰC HÀNH 4 XỬ LÝ TÍN HIỆU SỐ

Trần Minh Quang – 2412855

23/10/2026

Tóm tắt nội dung

Tổng Quan: Bài thực hành 4 gồm ba bài tính biến đổi Z và miền hội tụ (ROC), hai bài tìm hiểu thư viện xử lý âm thanh và ảnh trong Scilab kèm demo trên dữ liệu thật, và hai bài về tích chập tuyến tính, tích chập vòng bằng phương pháp gập–dịch và phương pháp ma trận. Kết quả mô phỏng trên Scilab đều khớp với tính toán giải tích.

1 Giới Thiệu

Biến đổi Z là công cụ chính để phân tích tín hiệu và hệ thống rời rạc: nó biến phép tích chập trong miền thời gian thành phép nhân đa thức, và miền hội tụ cho biết tín hiệu là nhân quả, phản nhân quả hay hai phía. Bên cạnh phần lý thuyết, bài thực hành này cho em làm quen với việc xử lý dữ liệu thật trong Scilab (một đoạn âm thanh và một bức ảnh), đồng thời mở rộng hàm tích chập đã viết ở Lab 3 sang phương pháp ma trận và tích chập vòng. Công cụ sử dụng là Scilab 2026.1.0.

2 Mục Tiêu

- **Về lý thuyết:** tính được biến đổi Z và xác định ROC của tín hiệu hữu hạn, nhân quả và hai phía; hiểu điều kiện hội tụ của tổng tích chập và ý nghĩa của tích chập vòng.
- **Về kỹ năng:** tìm hiểu các hàm và toolbox xử lý âm thanh, ảnh có trong Scilab; hiện thực tích chập tuyến tính và tích chập vòng bằng hai cách khác nhau.
- **Về trực quan hóa:** vẽ dạng sóng, phổ biên độ, ảnh và histogram; vẽ tín hiệu vào, ra và năng lượng trên cùng một figure.

3 Phương Pháp

- **Bài 1–3:** tính trực tiếp từ định nghĩa $X(z) = \sum_{n=-\infty}^{\infty} x(n)z^{-n}$, dùng tổng cấp số nhân để tìm dạng đóng và điều kiện hội tụ của từng thành phần; ROC của tổng là giao các ROC thành phần.

- **Bài 4–5:** tìm hiểu các hàm có sẵn và toolbox (qua ATOMS), sau đó viết script demo. Dữ liệu dùng là đoạn âm thanh tiếng mèo kêu grừ dài 4 giây (nguồn: pixabay.com) và ảnh chú chó nhà em, được thu nhỏ còn 384×512 pixel.
- **Bài 6–7:** tính tay bằng bảng gập–dịch và bằng phép nhân ma trận, sau đó hiện thực cả hai cách trên Scilab, đối chiếu với nhau và với hàm `conv` có sẵn.

Các tín hiệu hữu hạn được biểu diễn giống Lab 3: một vector giá trị và chỉ số gốc tọa độ, phần tử thứ i ứng với thời điểm $n = i - \text{origin}$.

4 Nội Dung Thực Hành

4.1 Bài tập 1 – Biến đổi Z của tín hiệu hữu hạn

Bài tập 1. Tìm biến đổi Z và miền hội tụ (ROC) của tín hiệu

$$x(n) = 2\delta(n+2) - \delta(n+1) + 2\delta(n) - 3\delta(n-1) + 4\delta(n-2).$$

Cơ sở lý thuyết và tính toán:

Áp dụng cặp biến đổi $\delta(n - n_0) \overset{Z}{\longleftrightarrow} z^{-n_0}$ (hội tụ mọi nơi trừ $z = 0$ khi $n_0 > 0$ và trừ $z = \infty$ khi $n_0 < 0$) cùng tính tuyến tính:

$$X(z) = 2z^2 - z + 2 - 3z^{-1} + 4z^{-2}. \tag{1}$$

Tín hiệu có chiều dài hữu hạn nên tổng luôn hội tụ, ngoại trừ hai điểm: các số hạng z^2, z (ứng với $n < 0$) tiến ra vô cùng khi $z \to \infty$, còn các số hạng z^{-1}, z^{-2} (ứng với $n > 0$) tiến ra vô cùng khi $z \to 0$. Do đó

$$\text{ROC: } 0 < |z| < \infty.$$

4.2 Bài tập 2 – Tổng hai tín hiệu nhân quả

Bài tập 2. Tìm biến đổi Z và ROC của $x(n) = 0.5^n u(n) + 0.4^n u(n)$.

Cơ sở lý thuyết và tính toán:

Với tín hiệu nhân quả $a^n u(n)$:

$$\sum_{n=0}^{\infty} a^n z^{-n} = \sum_{n=0}^{\infty} \left(az^{-1}\right)^n = \frac{1}{1 - az^{-1}}, \qquad \text{hội tụ khi } |az^{-1}| < 1 \iff |z| > |a|.$$

Áp dụng cho từng thành phần:

$$0.5^n u(n) \overset{Z}{\longleftrightarrow} \frac{1}{1 - 0.5z^{-1}},\ |z| > 0.5; \qquad 0.4^n u(n) \overset{Z}{\longleftrightarrow} \frac{1}{1 - 0.4z^{-1}},\ |z| > 0.4.$$

Cộng lại và quy đồng:

$$X(z) = \frac{1}{1-0.5z^{-1}} + \frac{1}{1-0.4z^{-1}} = \frac{2-0.9z^{-1}}{(1-0.5z^{-1})(1-0.4z^{-1})}. \tag{2}$$

ROC là giao của hai miền: $|z| > 0.5$. Đây là phần ngoài của đường tròn đi qua cực có bán kính lớn nhất, đúng với tính chất của tín hiệu nhân quả. Hệ có hai cực tại $z = 0.5$, $z = 0.4$ và hai không điểm tại $z = 0$, $z = 0.45$.

4.3 Bài tập 3 – Tín hiệu hai phía

Bài tập 3. Tìm biến đổi Z và ROC của $x(n) = 0.5^n u(n) + 0.9^n u(-n-1)$.

Cơ sở lý thuyết và tính toán:

Thành phần thứ nhất là tín hiệu nhân quả, tương tự Bài tập 2:

$$0.5^n u(n) \xleftrightarrow{Z} \frac{1}{1-0.5z^{-1}}, \qquad |z| > 0.5.$$

Thành phần thứ hai là tín hiệu *phản nhân quả* (chỉ khác 0 khi $n \le -1$). Đặt $m = -n$:

$$\sum_{n=-\infty}^{-1} 0.9^n z^{-n} = \sum_{m=1}^{\infty} \left(\frac{z}{0.9}\right)^m = \frac{z/0.9}{1-z/0.9} = \frac{z}{0.9-z} = -\frac{1}{1-0.9z^{-1}},$$

hội tụ khi $|z/0.9| < 1 \iff |z| < 0.9$. Cộng hai thành phần:

$$X(z) = \frac{1}{1-0.5z^{-1}} - \frac{1}{1-0.9z^{-1}} = \frac{-0.4\,z^{-1}}{(1-0.5z^{-1})(1-0.9z^{-1})}. \tag{3}$$

ROC là giao của hai miền, tức vành khuyên

$$\text{ROC: } 0.5 < |z| < 0.9.$$

Vì $0.5 < 0.9$ nên vành khuyên này khác rỗng và biến đổi Z tồn tại. Đây là đặc trưng của tín hiệu hai phía: ROC nằm giữa hai cực, cực 0.5 thuộc phần nhân quả ở phía trong, cực 0.9 thuộc phần phản nhân quả ở phía ngoài. Nếu đổi chỗ hai cơ số (phần nhân quả có cơ số lớn hơn) thì giao sẽ rỗng và $X(z)$ không tồn tại.

4.4 Bài tập 4 – Thư viện xử lý âm thanh trong Scilab

Bài tập 4. Tìm hiểu các thư viện hoặc toolbox có sẵn trong Scilab để xử lý dữ liệu âm thanh.

1. Tìm hiểu thư viện:

Scilab có sẵn các hàm xử lý âm thanh ngay trong bản cài đặt chuẩn, không cần cài thêm toolbox:

Hàm	Chức năng
`wavread / wavwrite`	Đọc / ghi file `.wav`
`sound / playsnd`	Phát âm thanh ra loa
`analyze, mapsound`	Vẽ phổ tần số, spectrogram
`fft`	Tính phổ của tín hiệu

Bảng 1: Một số hàm xử lý âm thanh có sẵn trong Scilab

2. Demo và mã nguồn:

Em dùng đoạn âm thanh tiếng mèo kêu grừ dài 4 giây (nguồn: https://pixabay.com), đã chuyển từ `.mp3` sang `.wav` vì `wavread` chỉ đọc được file `.wav`. Script đọc file, vẽ dạng sóng theo thời gian và phổ biên độ tính bằng `fft`.

```
clc; clear;

// Cac ham xu ly am thanh co san trong Scilab (khong can cai toolbox):
//   wavread / wavwrite  : doc / ghi file .wav
//   loadwave / savewave : doc / ghi file .wav (dang cu)
//   auread / auwrite    : doc / ghi file .au
//   sound / playsnd     : phat am thanh ra loa
//   analyze             : ve pho tan so cua tin hieu
//   mapsound            : ve spectrogram (pho theo thoi gian)
//   lin2mu / mu2lin     : nen / giai nen mu-law

function [y, fs]=load_audio(wav_path)
    // doc file .wav, neu khong co file thi tao tin hieu thu
    if wav_path <> "" then
        [y, fs, bits] = wavread(wav_path);
        disp(bits, "so bit moi mau =");
    else
        // tin hieu thu: tong 2 song sin 440 Hz va 880 Hz trong 1 giay
        fs = 8000;
        t = 0:1/fs:1 - 1/fs;
        y = 0.5*sin(2*%pi*440*t) + 0.3*sin(2*%pi*880*t);

        // ghi ra file .wav roi doc lai de thu wavwrite / wavread
        tmp_file = TMPDIR + "/ex4_4.wav";
        wavwrite(y, fs, tmp_file);
        [y, fs] = wavread(tmp_file);
    end

    // chi lay kenh dau tien va dua ve vector hang
    if size(y, 1) > 1 & size(y, 2) > 1 then
        if size(y, 1) < size(y, 2) then
            y = y(1, :);
        else
            y = y(:, 1)';
```

```
        end
    end
    y = matrix(y, 1, -1);
endfunction

// Duong dan file .wav, tinh tu thu muc chua script (de trong "" de dung tin hieu thu)
//   cat_purr.wav : tieng meo grru 4 giay, mono 16 kHz (nguon: pixabay.com)
src_dir = get_absolute_file_path("Ex_4.4.sci");
wav_path = src_dir + "../data/cat_purr.wav";

[y, fs] = load_audio(wav_path);
N = length(y);
disp(fs, "tan so lay mau fs =");
disp(N / fs, "thoi luong (s) =");

// Truc thoi gian va pho bien do (FFT)
t = (0:N - 1) / fs;
Y = abs(fft(y)) / N;
f = (0:N - 1) * fs / N;

// gioi han tan so hien thi (nang luong tieng grru tap trung o tan so thap)
f_show = min(fs / 2, 1000);
half = find(f <= f_show);

clf();

// dang song theo thoi gian (toan bo doan am thanh)
subplot(2, 1, 1);
plot(t, y, 'b');
xgrid(color('gray'));
title("$\text{Audio waveform } x(t)$", "fontsize", 3);
xlabel("$\text{Time (s)}$", "fontsize", 3);
ylabel("$\text{Amplitude}$", "fontsize", 3);

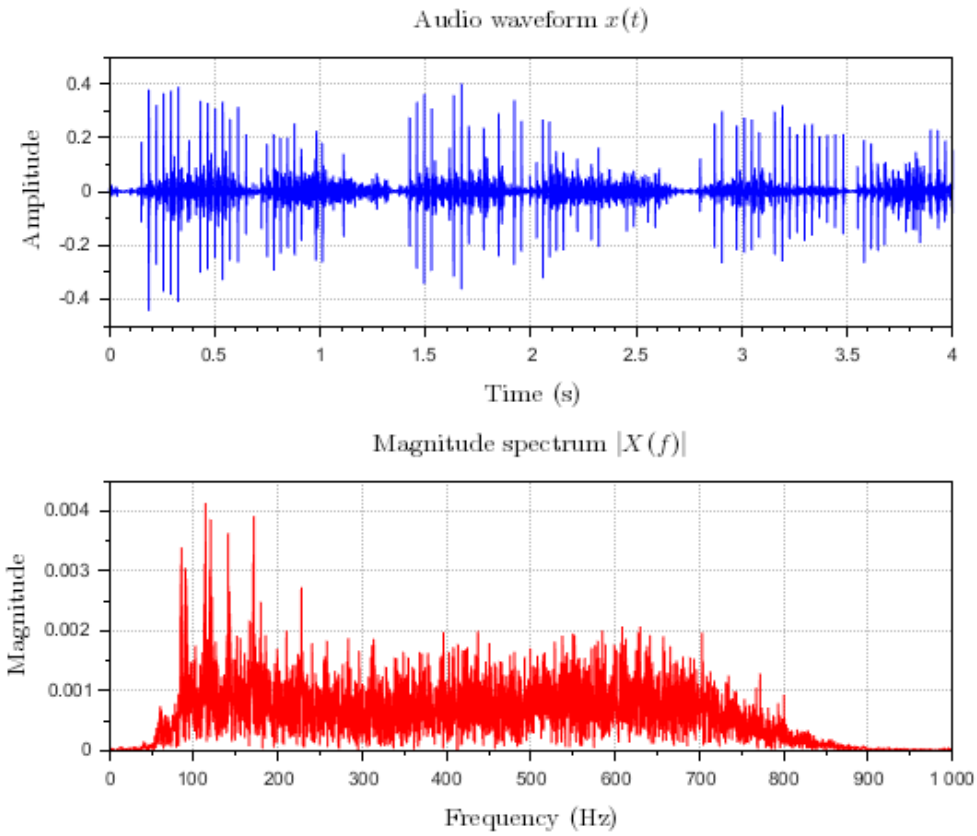
// pho bien do (tu 0 den f_show)
subplot(2, 1, 2);
plot(f(half), 2 * Y(half), 'r');
xgrid(color('gray'));
title("$\text{Magnitude spectrum } |X(f)|$", "fontsize", 3);
xlabel("$\text{Frequency (Hz)}$", "fontsize", 3);
ylabel("$\text{Magnitude}$", "fontsize", 3);

// bo comment dong duoi de nghe thu am thanh
// playsnd(y, fs);
```

3. Kết quả và nhận xét:

Scilab 2026.1.0 Console

```
--> exec('Ex_4.4.sci', -1)
   16.
  "so bit moi mau ="
   16000.
  "tan so lay mau fs ="
   4.
  "thoi luong (s) ="
```



Hình 1: Dạng sóng và phổ biên độ của đoạn âm thanh tiếng mèo kêu grừ

File âm thanh được đọc đúng: 16 bit mỗi mẫu, tần số lấy mẫu 16 kHz, dài 4 giây. Dạng sóng (Hình 1, trên) gồm nhiều xung ngắn lặp lại đều đặn, đúng với đặc điểm của tiếng grừ. Phổ biên độ (dưới) cho thấy năng lượng tập trung ở vùng tần số thấp, khoảng 80–800 Hz, với đỉnh lớn nhất quanh 114 Hz.

4.5 Bài tập 5 – Thư viện xử lý ảnh trong Scilab

Bài tập 5. Tìm hiểu các thư viện hoặc toolbox xử lý ảnh trong Scilab. Sau đó thực hiện một ví dụ hoặc demo minh họa một thao tác đơn giản (ví dụ: hiển thị histogram, cân bằng histogram, làm mờ, đóng dấu watermark) trên một ảnh số.

1. Tìm hiểu thư viện:

Scilab chuẩn không có hàm đọc ảnh, cần cài thêm toolbox qua ATOMS. Toolbox phổ biến hiện nay là **IPCV**, cài bằng `atomsInstall("IPCV")`, cung cấp các hàm `imread`, `imshow`, `imhist`, `imfilter`, ...; toolbox cũ hơn là **SIVP** nhưng không còn được cập nhật. Để script chạy được cả khi chưa cài toolbox, em tự viết hàm `read_pgm` đọc ảnh xám dạng `.pgm`, còn các phép xử lý chỉ dùng hàm có sẵn của Scilab (`conv2`, `cumsum`, `Matplot`).

2. Các phép xử lý thực hiện:

- **Histogram:** đếm số pixel ở mỗi mức xám từ 0 đến 255.
- **Cân bằng histogram:** ánh xạ mức xám qua hàm phân bố tích lũy để trải đều các mức xám, giúp tăng độ tương phản.
- **Làm mờ:** tích chập ảnh với bộ lọc trung bình 5×5.
- **Watermark:** giấu một ảnh nhị phân vào bit thấp nhất (LSB) của mỗi pixel, sau đó tách lại.

3. Mã nguồn thực thi trên Scilab:

```
clc; clear;

// Cac toolbox xu ly anh cho Scilab (cai qua ATOMS):
//   IPCV : Image Processing and Computer Vision, dang duoc bao tri
//          cai dat: atomsInstall("IPCV"), sau do khoi dong lai Scilab
//          ham hay dung: imread, imshow, imwrite, rgb2gray, imhist,
    ↪ imfilter, imresize
//   SIVP : toolbox cu, khong con duoc cap nhat cho Scilab moi
// Neu chua cai IPCV, script doc anh xam dang .pgm bang ham tu viet (
    ↪ read_pgm),
// cac phep xu ly ben duoi chi dung ham co san cua Scilab (conv2, cumsum,
    ↪ Matplot)

function img=read_pgm(pgm_path)
    // doc anh xam dinh dang PGM nhi phan (P5) ma khong can toolbox
    fd = mopen(pgm_path, "rb");
    info = fileinfo(pgm_path);
    b = mget(info(1), "uc", fd);
    mclose(fd);

    // tach 4 truong header: "P5", chieu rong, chieu cao, gia tri max
    tokens = [];
    pos = 1;
    while size(tokens, "*") < 4
        if b(pos) == 35 then
            // bo qua dong comment bat dau bang '#'
            while b(pos) <> 10
                pos = pos + 1;
            end
        elseif or(b(pos) == [9, 10, 13, 32]) then
            pos = pos + 1;
```

```
        else
            p0 = pos;
            while ~or(b(pos) == [9, 10, 13, 32])
                pos = pos + 1;
            end
            tokens = [tokens, ascii(b(p0:pos - 1))];
        end
    end
    if tokens(1) <> "P5" then
        error("read_pgm: chi ho tro PGM nhi phan (P5)");
    end
    w = evstr(tokens(2));
    h = evstr(tokens(3));

    // du lieu pixel bat dau sau 1 ky tu trang ke tu header, luu theo tung hang
    data = b(pos + 1:pos + w * h);
    img = matrix(data, w, h)';
endfunction

function img=load_gray_image(img_path)
    // doc anh .pgm bang read_pgm, anh khac thi doc bang IPCV (neu co), nguoc lai tao anh thu
    if img_path <> "" & convstr(fileext(img_path)) == ".pgm" then
        img = read_pgm(img_path);
    elseif img_path <> "" & exists("imread") then
        im = double(imread(img_path));
        if ndims(im) == 3 then
            // chuyen anh mau sang anh xam
            img = 0.299*im(:, :, 1) + 0.587*im(:, :, 2) + 0.114*im(:, :, 3) ;
        else
            img = im;
        end
    else
        // anh thu 128x128: nen gradient do tuong phan thap + hinh tron sang
        [c, r] = meshgrid(1:128, 1:128);
        img = 70 + 40 * c / 128;
        img(((r - 64).^2 + (c - 64).^2) < 30^2) = 140;
    end
    img = round(img);
endfunction

function h=gray_hist(img)
    // dem so pixel cho tung muc xam 0..255
    h = zeros(1, 256);
    for v = 0:255
```

```
        h(v + 1) = sum(img == v);
    end
endfunction

function out=hist_equalize(img)
    // can bang histogram: anh xa muc xam qua ham phan bo tich luy (CDF)
    h = gray_hist(img);
    cdf = cumsum(h) / sum(h);
    map = round(255 * cdf);
    out = matrix(map(img(:) + 1), size(img));
endfunction

function out=blur(img, w)
    // lam mo bang bo loc trung binh kich thuoc w x w
    kernel = ones(w, w) / (w * w);
    out = round(conv2(img, kernel, "same"));
endfunction

function [marked, extracted]=lsb_watermark(img, wm)
    // giau watermark nhi phan (0/1) vao bit thap nhat (LSB) cua anh
    marked = img - modulo(img, 2) + wm;
    // tach watermark ra lai tu bit thap nhat
    extracted = modulo(marked, 2);
endfunction

function show_img(img, ttl)
    // hien thi anh xam (gia tri 0..255) bang Matplot
    Matplot(img + 1);
    ax = gca();
    ax.isoview = "on";
    ax.axes_visible = ["off", "off", "off"];
    title(ttl, "fontsize", 3);
endfunction

// Duong dan anh, tinh tu thu muc chua script (de trong "" de dung anh thu)
//   dog.pgm : anh xam, doc duoc ngay khong can IPCV
//   dog.jpg : anh mau, can cai IPCV
src_dir = get_absolute_file_path("Ex_4.5.sci");
img_path = src_dir + "../data/dog.pgm";

img = load_gray_image(img_path);
[nr, nc] = size(img);

// Watermark: khung chu nhat o giua anh
wm = zeros(nr, nc);
r1 = round(nr/4); r2 = round(3*nr/4);
c1 = round(nc/4); c2 = round(3*nc/4);
wm(r1:r2, [c1:c1+3, c2-3:c2]) = 1;
```

```
wm([r1:r1+3, r2-3:r2], c1:c2) = 1;

// Thuc hien cac phep xu ly
img_eq = hist_equalize(img);
img_blur = blur(img, 5);
[img_marked, wm_out] = lsb_watermark(img, wm);

disp(max(abs(img_marked - img)), "sai lech lon nhat sau khi giau watermark =");
disp(and(wm_out == wm), "tach watermark dung =");

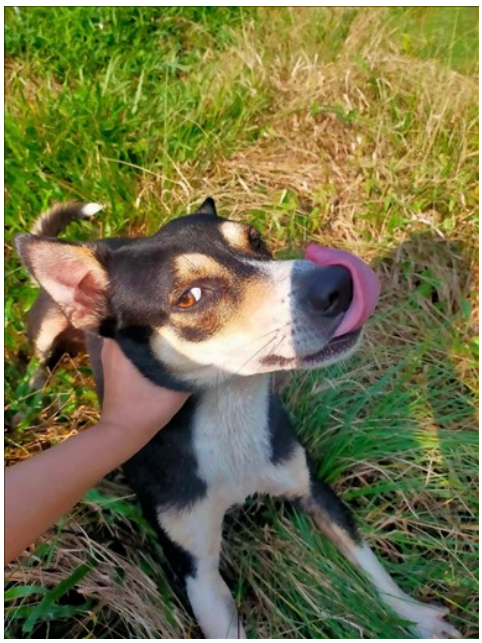
// Hien thi anh
f1 = scf(0);
clf(f1);
f1.color_map = gray(256);
subplot(2, 3, 1); show_img(img, "$\text{Original}$");
subplot(2, 3, 2); show_img(img_eq, "$\text{Histogram equalized}$");
subplot(2, 3, 3); show_img(img_blur, "$\text{Blur } 5 \times 5$");
subplot(2, 3, 4); show_img(img_marked, "$\text{Watermarked (LSB)}$");
subplot(2, 3, 5); show_img(255 * wm_out, "$\text{Extracted watermark}$");

// Hien thi histogram truoc va sau khi can bang
f2 = scf(1);
clf(f2);
h_orig = gray_hist(img);
h_eq = gray_hist(img_eq);
h_max = max([h_orig, h_eq]) * 1.1;

// ve dang thanh (plot2d3) thay vi bar de truc hoanh 256 muc xam khong bi dinh chu
subplot(2, 1, 1);
plot2d3(0:255, h_orig, 2);
ax1 = gca();
ax1.data_bounds = [0, 0; 255, h_max];
ax1.tight_limits = "on";
xgrid(color('gray'));
title("$\text{Histogram of original image}$", "fontsize", 3);
ylabel("$\text{Pixel count}$", "fontsize", 3);

subplot(2, 1, 2);
plot2d3(0:255, h_eq, 5);
ax2 = gca();
ax2.data_bounds = [0, 0; 255, h_max];
ax2.tight_limits = "on";
xgrid(color('gray'));
title("$\text{Histogram after equalization}$", "fontsize", 3);
xlabel("$\text{Gray level}$", "fontsize", 3);
ylabel("$\text{Pixel count}$", "fontsize", 3);
```

4. Kết quả và nhận xét:



Hình 2: Ảnh gốc dùng cho demo (Credit: Chó ở nhà em)

Scilab 2026.1.0 Console

```
--> exec('Ex_4.5.sci', -1)
   1.
  "sai lech lon nhat sau khi giau watermark ="
  T
  "tach watermark dung ="
```



(a) Ảnh xám gốc



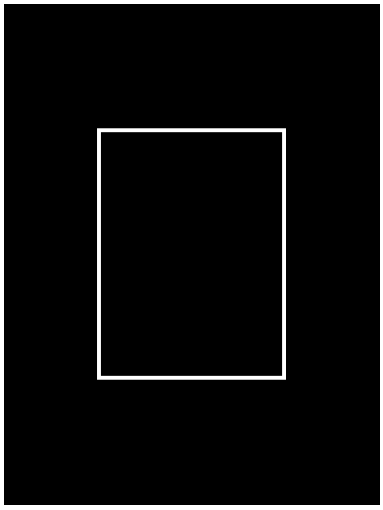
(b) Cân bằng histogram



(c) Làm mờ 5×5

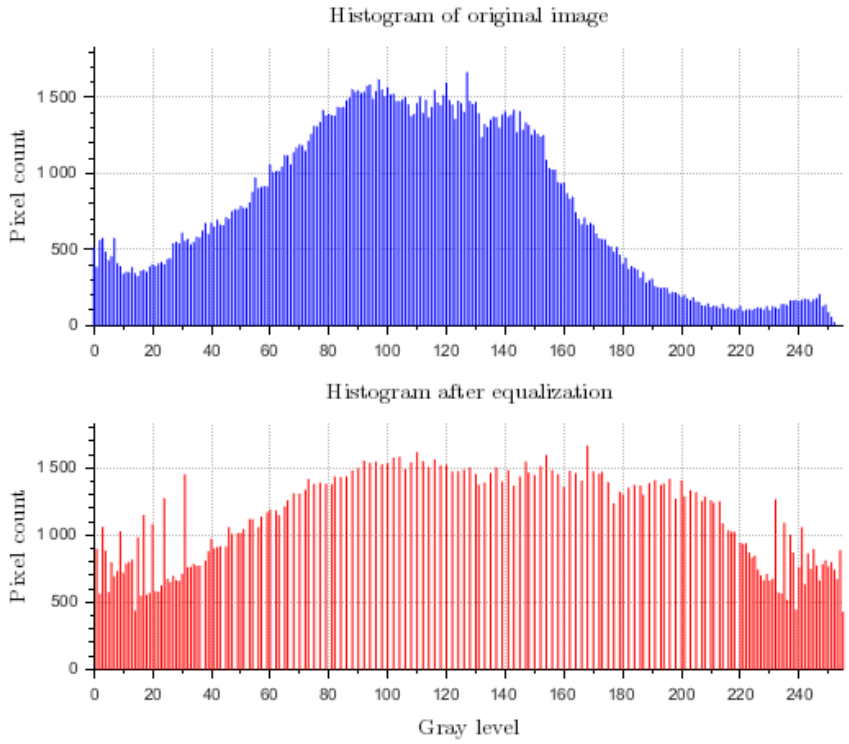


(d) Ảnh đã giấu watermark



(e) Watermark tách ra

Hình 3: Ảnh xám gốc và kết quả các phép xử lý



Hình 4: Histogram của ảnh trước (trên) và sau (dưới) khi cân bằng

Nhận xét:

- **Cân bằng histogram:** histogram ảnh gốc dồn chủ yếu ở các mức xám trung bình, sau khi cân bằng thì trải đều hơn trên toàn dải 0–255 (Hình 4). Ảnh (b) có độ tương phản cao hơn, cỏ và lông đen rõ chi tiết hơn so với ảnh gốc (a).
- **Làm mờ:** ở ảnh (c), các chi tiết nhỏ như sợi cỏ, ria mép bị nhòe đi, còn các vùng lớn vẫn giữ hình dạng.
- **Watermark:** ảnh (d) nhìn không khác gì ảnh gốc vì mỗi pixel chỉ thay đổi tối đa 1 mức xám (console in ra 1), và watermark tách ra (e) đúng với watermark đã giấu (console in `T`).

4.6 Bài tập 6 – Tích chập tuyến tính

Bài tập 6. Cho hai tín hiệu rời rạc $x(n)$, $h(n)$, $n = 0, \pm 1, \pm 2, \ldots$ xác định trên toàn trục n. Tích chập tuyến tính của chúng là

$$y(n) = \sum_{k=-\infty}^{\infty} h(k)\, x(n-k).$$

- Suy ra biểu thức của $y(n)$ sao cho $y(n)$ hội tụ trong các trường hợp: (i) cả $h(n)$ và $x(n)$ đều nhân quả; (ii) hệ có đáp ứng xung hữu hạn, $h(n) = 0$ với $n < 0$ hoặc $n > M$.

- Giả sử trường hợp (ii) xảy ra, tính $y(n)$ với $x(n) = [\underset{\uparrow}{1}, 2, -3, 2, 1]$ và $h(n) = [\underset{\uparrow}{1}, 0, -1]$ bằng phương pháp gập và dịch, hiện thực bằng Scilab.
- Tính lại $y(n)$ bằng phương pháp ma trận, hiện thực bằng Scilab.
- Vẽ $x(n)$, $y(n)$ và năng lượng của các tín hiệu trên cùng một hình.

1. Cơ sở lý thuyết và tính toán:

(i) Cả $h(n)$ và $x(n)$ đều nhân quả. Vì $h(k) = 0$ với $k < 0$ và $x(n-k) = 0$ khi $n - k < 0$, tức $k > n$, nên chỉ các số hạng $0 \le k \le n$ còn lại:

$$y(n) = \begin{cases} \displaystyle\sum_{k=0}^{n} h(k)\, x(n-k), & n \ge 0, \\ 0, & n < 0. \end{cases} \tag{4}$$

Với mỗi n, tổng chỉ có $n + 1$ số hạng hữu hạn nên $y(n)$ luôn xác định (hội tụ), và $y(n)$ cũng là tín hiệu nhân quả.

(ii) Hệ FIR. Vì $h(k) = 0$ ngoài đoạn $0 \le k \le M$:

$$y(n) = \sum_{k=0}^{M} h(k)\, x(n-k). \tag{5}$$

Mỗi $y(n)$ chỉ là tổng của $M + 1$ số hạng nên luôn hội tụ với mọi n. Hơn nữa, nếu $x(n)$ bị chặn, $|x(n)| \le B$, thì $|y(n)| \le B \sum_{k=0}^{M} |h(k)| < \infty$, nghĩa là hệ FIR luôn ổn định BIBO.

Phương pháp gập và dịch. Ở đây $h(n) = \delta(n) - \delta(n-2)$, $M = 2$, nên $y(n) = x(n) - x(n-2)$. Thực hiện theo các bước: gập $h(k)$ thành $h(-k) = [-1, 0, \underset{\uparrow}{1}]$, dịch đi n mẫu thành $h(n-k)$, nhân từng mẫu với $x(k)$ rồi cộng lại. Tín hiệu ra có chiều dài $L_y = 5 + 3 - 1 = 7$ và bắt đầu tại $n = 0 + 0 = 0$:

n	Các mẫu $x(k)$ chồng lên $h(n-k)$	$y(n)$
0	$x(0)h(0) = 1 \cdot 1$	1
1	$x(1)h(0) + x(0)h(1) = 2 + 0$	2
2	$x(2)h(0) + x(1)h(1) + x(0)h(2) = -3 + 0 - 1$	-4
3	$x(3)h(0) + x(2)h(1) + x(1)h(2) = 2 + 0 - 2$	0
4	$x(4)h(0) + x(3)h(1) + x(2)h(2) = 1 + 0 + 3$	4
5	$x(4)h(1) + x(3)h(2) = 0 - 2$	-2
6	$x(4)h(2) = -1$	-1

Bảng 2: Tính tích chập tuyến tính bằng phương pháp gập và dịch

Vậy $y(n) = [\underset{\uparrow}{1}, 2, -4, 0, 4, -2, -1]$.

Phương pháp ma trận. Viết $\mathbf{y} = \mathbf{H}\mathbf{x}$, trong đó $\mathbf{H}$ là ma trận Toeplitz kích thước $L_y \times L_x = 7 \times 5$. Cột thứ j của $\mathbf{H}$ là $h(n)$ dịch xuống $j-1$ mẫu:

$$\mathbf{y} = \begin{bmatrix} 1 & 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 \\ -1 & 0 & 1 & 0 & 0 \\ 0 & -1 & 0 & 1 & 0 \\ 0 & 0 & -1 & 0 & 1 \\ 0 & 0 & 0 & -1 & 0 \\ 0 & 0 & 0 & 0 & -1 \end{bmatrix} \begin{bmatrix} 1 \\ 2 \\ -3 \\ 2 \\ 1 \end{bmatrix} = \begin{bmatrix} 1 \\ 2 \\ -4 \\ 0 \\ 4 \\ -2 \\ -1 \end{bmatrix}.$$

Kết quả trùng với phương pháp gập và dịch.

Năng lượng. Với $E = \sum_n |x(n)|^2$:

$$E_x = 1+4+9+4+1 = 19, \qquad E_y = 1+4+16+0+16+4+1 = 42.$$

2. Mã nguồn thực thi trên Scilab:

```
clc; clear;

function [yn, yorigin]=conv_fold_shift(xn, xorigin, hn, horigin)
    // Truc thoi gian cho x(n) va h(n)
    nx = (1:length(xn)) - xorigin;
    nh = (1:length(hn)) - horigin;

    // Mien xac dinh cua y(n): tu nx_dau + nh_dau den nx_cuoi + nh_cuoi
    ny = (nx(1) + nh(1)):(nx($) + nh($));
    yn = zeros(1, length(ny));

    // Gap h(k) thanh h(-k)
    hf = hn($:-1:1);
    nhf = -nh($:-1:1);

    // Voi moi n: dich h(-k) di n mau -> h(n - k), nhan voi x(k) roi cong
    ↪ lai
    for i = 1:length(ny)
        n = ny(i);
        nshift = nhf + n;
        for j = 1:length(nshift)
            idx = find(nx == nshift(j));
            if ~isempty(idx) then
                yn(i) = yn(i) + xn(idx) * hf(j);
            end
        end
    end

    // Goc toa do moi
```

```
    yorigin = 1 - ny(1);
endfunction

function [yn, yorigin]=conv_matrix(xn, xorigin, hn, horigin)
    // Chieu dai cua x(n), h(n) va y(n)
    Lx = length(xn);
    Lh = length(hn);
    Ly = Lx + Lh - 1;

    // Ma tran Toeplitz H (Ly x Lx): cot thu j la h(n) dich xuong j - 1 mau
    H = zeros(Ly, Lx);
    for j = 1:Lx
        H(j:j + Lh - 1, j) = hn(:);
    end

    // y = H.x
    yn = (H * xn(:))';

    // Goc toa do moi
    yorigin = xorigin + horigin - 1;
endfunction

function E=energy(xn)
    // Nang luong tin hieu roi rac: E = sum |x(n)|^2
    E = sum(abs(xn).^2);
endfunction

xn = [1, 2, -3, 2, 1];
xorigin = 1;
hn = [1, 0, -1];
horigin = 1;

// 6.3 phuong phap gap va dich
[yn1, yorigin1] = conv_fold_shift(xn, xorigin, hn, horigin);
disp(yn1, "yn (gap va dich) =");
disp(yorigin1, "yorigin =");

// 6.4 phuong phap ma tran
[yn2, yorigin2] = conv_matrix(xn, xorigin, hn, horigin);
disp(yn2, "yn (ma tran) =");
disp(yorigin2, "yorigin =");

// kiem tra voi ham conv co san cua Scilab
disp(conv(xn, hn), "conv(xn, hn) =");

// 6.5 nang luong cua x(n) va y(n)
Ex = energy(xn);
Ey = energy(yn1);
```

```
disp(Ex, "Ex =");
disp(Ey, "Ey =");

// Truc thoi gian cho x(n) va y(n)
nx = (1:length(xn)) - xorigin;
ny = (1:length(yn1)) - yorigin1;

// gioi han hien thi truc hoanh va truc tung chung
n_min = min([nx, ny]) - 1;
n_max = max([nx, ny]) + 1;
y_min = min([xn, yn1, 0]) - 1;
y_max = max([xn, yn1, 0]) + 1;

clf();

// x(n)
subplot(3, 1, 1);
ax1 = gca();
ax1.x_location = "origin";
ax1.box = "off";
plot2d3(nx, xn, 2);
plot(nx, xn, 'bo');
// dat gioi han truc sau khi ve de Scilab khong tu co gian lai
ax1.data_bounds = [n_min, y_min; n_max, y_max];
ax1.tight_limits = "on";
ax1.x_ticks = tlist(["ticks", "locations", "labels"], n_min:n_max, string(
   ↪ n_min:n_max));
xgrid(color('gray'));
title("$\text{Input signal } x(n)$", "fontsize", 3);
ylabel("$\text{Amplitude}$", "fontsize", 3);

// y(n) = x(n) * h(n)
subplot(3, 1, 2);
ax2 = gca();
ax2.x_location = "origin";
ax2.box = "off";
plot2d3(ny, yn1, 5);
plot(ny, yn1, 'ro');
ax2.data_bounds = [n_min, y_min; n_max, y_max];
ax2.tight_limits = "on";
ax2.x_ticks = tlist(["ticks", "locations", "labels"], n_min:n_max, string(
   ↪ n_min:n_max));
xgrid(color('gray'));
title("$\text{Linear convolution } y(n) = x(n) * h(n)$", "fontsize", 3);
xlabel("$\text{Sample index } n$", "fontsize", 3);
ylabel("$\text{Amplitude}$", "fontsize", 3);

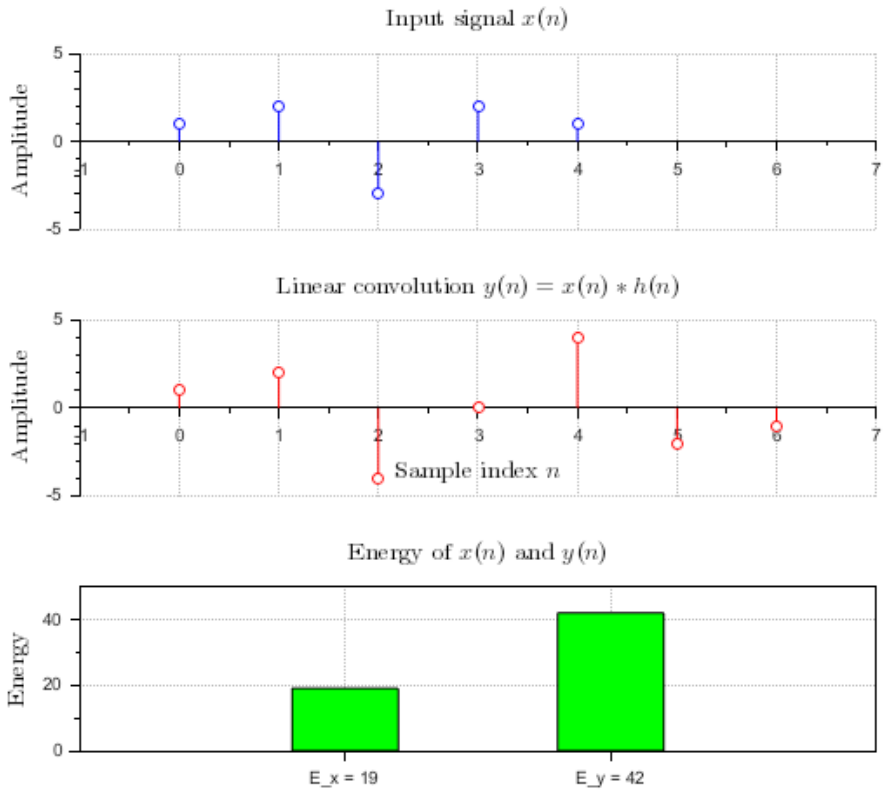
// nang luong E_x va E_y
```

```
subplot(3, 1, 3);
bar([1, 2], [Ex, Ey], 0.4, 'green');
ax3 = gca();
ax3.data_bounds = [0, 0; 3, max([Ex, Ey]) + 5];
ax3.x_ticks = tlist(["ticks", "locations", "labels"], [1, 2], ["E_x = " +
    ↪ string(Ex), "E_y = " + string(Ey)]);
xgrid(color('gray'));
title("$\text{Energy of } x(n) \text{ and } y(n)$", "fontsize", 3);
ylabel("$\text{Energy}$", "fontsize", 3);
```

3. Kết quả mô phỏng và nhận xét:

Scilab 2026.1.0 Console

```
--> exec('Ex_4.6.sci', -1)
   1.   2.  -4.   0.   4.  -2.  -1.
  "yn (gap va dich) ="
   1.
  "yorigin ="
   1.   2.  -4.   0.   4.  -2.  -1.
  "yn (ma tran) ="
   1.
  "yorigin ="
   1.   2.  -4.   0.   4.  -2.  -1.
  "conv(xn, hn) ="
   19.
  "Ex ="
   42.
  "Ey ="
```



Hình 5: Tín hiệu vào $x(n)$, tín hiệu ra $y(n) = x(n) * h(n)$ và năng lượng của hai tín hiệu

Hai phương pháp và hàm `conv` có sẵn của Scilab cho cùng kết quả $[1, 2, -4, 0, 4, -2, -1]$, với gốc tọa độ ở phần tử đầu tiên. Trên Hình 5, $y(n)$ dài 7 mẫu, kéo dài thêm $M = 2$ mẫu so với $x(n)$. Năng lượng tăng từ $E_x = 19$ lên $E_y = 42$, vì hệ $h(n) = \delta(n) - \delta(n-2)$ cộng thêm một bản trễ đảo dấu của $x(n)$ vào chính nó. Phương pháp gập–dịch phù hợp để hiểu bản chất và tính tay, còn phương pháp ma trận thuận tiện khi lập trình vì chỉ là một phép nhân ma trận–vector.

4.7 Bài tập 7 – Tích chập vòng

Bài tập 7. Với tín hiệu tuần hoàn $x(n)$ có chiều dài N và đáp ứng xung $h(n)$ có chiều dài M, tích chập vòng giữa $x(n)$ và $h(n)$ được định nghĩa là

$$y(n) = \sum_{k=0}^{M-1} h(k)\, x[(n-k) \bmod N].$$

Giải thích vì sao $y(n)$ có thể được suy ra như vậy.

- Tính $y(n)$ với $x(n) = [\underset{\uparrow}{1}, 2, -3, 2, 1]$ và $h(n) = [\underset{\uparrow}{1}, 0, -1, -1, 1]$ bằng phương pháp gập và dịch, hiện thực bằng Scilab.
- Tính lại $y(n)$ bằng phương pháp ma trận, hiện thực bằng Scilab.

- Vẽ $x(n)$, $y(n)$ và năng lượng của các tín hiệu trên cùng một hình.

1. Cơ sở lý thuyết và tính toán:

Giải thích công thức. Xuất phát từ tích chập tuyến tính $y(n) = \sum_k h(k)\, x(n-k)$:

- $h(n)$ chỉ khác 0 khi $0 \le k \le M-1$, nên tổng chỉ còn M số hạng, từ $k=0$ đến $M-1$.
- $x(n)$ tuần hoàn chu kỳ N, tức $x(n+N) = x(n)$ với mọi n. Vì vậy giá trị $x(n-k)$ tại một chỉ số bất kỳ bằng giá trị tại chỉ số tương ứng trong một chu kỳ cơ bản $[0, N-1]$, chính là $x[(n-k) \bmod N]$. Nói cách khác, khi dịch tín hiệu, mẫu nào "trượt" ra khỏi đoạn $[0, N-1]$ ở một đầu sẽ "quay vòng" vào lại ở đầu kia, đây là lý do gọi là tích chập *vòng*.
- Do $x(n)$ tuần hoàn chu kỳ N nên $y(n)$ cũng tuần hoàn chu kỳ N: $y(n+N) = \sum_k h(k)\, x(n+N-k) = y(n)$. Vì thế chỉ cần tính $y(n)$ cho $n = 0, 1, \ldots, N-1$.

Tích chập vòng N điểm cũng chính là phép toán ứng với tích hai DFT N điểm, $Y(k) = H(k)X(k)$, nên đây là nền tảng cho cách tính tích chập nhanh bằng FFT.

Phương pháp gập và dịch. Ở đây $N = M = 5$ và $h(n) = \delta(n) - \delta(n-2) - \delta(n-3) + \delta(n-4)$, nên

$$y(n) = x(n) - x[(n-2) \bmod 5] - x[(n-3) \bmod 5] + x[(n-4) \bmod 5], \qquad n = 0, \ldots, 4.$$

Với $x = [x_0, x_1, x_2, x_3, x_4] = [1, 2, -3, 2, 1]$:

n	$x(n) - x[(n-2)_5] - x[(n-3)_5] + x[(n-4)_5]$	$y(n)$
0	$x_0 - x_3 - x_2 + x_1 = 1 - 2 + 3 + 2$	4
1	$x_1 - x_4 - x_3 + x_2 = 2 - 1 - 2 - 3$	-4
2	$x_2 - x_0 - x_4 + x_3 = -3 - 1 - 1 + 2$	-3
3	$x_3 - x_1 - x_0 + x_4 = 2 - 2 - 1 + 1$	0
4	$x_4 - x_2 - x_1 + x_0 = 1 + 3 - 2 + 1$	3

Bảng 3: Tính tích chập vòng bằng phương pháp gập và dịch

Vậy $y(n) = [\underset{\uparrow}{4}, -4, -3, 0, 3]$. Kiểm tra nhanh: $\sum y(n) = 0 = (\sum h(n))\,(\sum x(n)) = 0 \cdot 3$, đúng.

Phương pháp ma trận. Viết $\mathbf{y} = \mathbf{C}\mathbf{h}$, với $\mathbf{C}$ là ma trận vòng (circulant) $N \times N$, phần tử $C_{n,k} = x[(n-k) \bmod N]$. Mỗi cột là $x(n)$ dịch vòng xuống thêm 1 mẫu:

$$\mathbf{y} = \begin{bmatrix} 1 & 1 & 2 & -3 & 2 \\ 2 & 1 & 1 & 2 & -3 \\ -3 & 2 & 1 & 1 & 2 \\ 2 & -3 & 2 & 1 & 1 \\ 1 & 2 & -3 & 2 & 1 \end{bmatrix} \begin{bmatrix} 1 \\ 0 \\ -1 \\ -1 \\ 1 \end{bmatrix} = \begin{bmatrix} 4 \\ -4 \\ -3 \\ 0 \\ 3 \end{bmatrix}.$$

Trường hợp $M < N$ thì chỉ cần bù 0 cho $h(n)$ cho đủ N mẫu.

Năng lượng. $E_x = 19$ và $E_y = 16 + 16 + 9 + 0 + 9 = 50$.

2. Mã nguồn thực thi trên Scilab:

```
clc; clear;

function yn=circonv_fold_shift(xn, hn)
    // Chu ky N cua x(n) va chieu dai M cua h(n)
    N = length(xn);
    M = length(hn);
    yn = zeros(1, N);

    // y(n) = sum_{k=0}^{M-1} h(k).x((n - k) mod N), n = 0..N-1
    // gap x(k) thanh x(-k), dich n mau; chi so vuot ra ngoai [0, N-1] thi
    ↪ quay vong lai
    for n = 0:N - 1
        for k = 0:M - 1
            yn(n + 1) = yn(n + 1) + hn(k + 1) * xn(pmodulo(n - k, N) + 1);
        end
    end
endfunction

function yn=circonv_matrix(xn, hn)
    // Chu ky N cua x(n) va chieu dai M cua h(n)
    N = length(xn);
    M = length(hn);

    // Ma tran vong C (N x N): moi cot la x(n) dich vong xuong k mau
    C = zeros(N, N);
    for k = 0:N - 1
        for n = 0:N - 1
            C(n + 1, k + 1) = xn(pmodulo(n - k, N) + 1);
        end
    end

    // Bu 0 cho h(n) neu M < N, sau do y = C.h
    hpad = zeros(N, 1);
    hpad(1:M) = hn(:);
    yn = (C * hpad)';
endfunction

function E=energy(xn)
    // Nang luong tin hieu roi rac: E = sum |x(n)|^2
    E = sum(abs(xn).^2);
endfunction

// Ca x(n) va h(n) deu co goc tai phan tu dau tien (n = 0)
xn = [1, 2, -3, 2, 1];
hn = [1, 0, -1, -1, 1];
```

```
45
46 // 7.2 phuong phap gap va dich
47 yn1 = circonv_fold_shift(xn, hn);
48 disp(yn1, "yn (gap va dich) =");
49
50 // 7.3 phuong phap ma tran
51 yn2 = circonv_matrix(xn, hn);
52 disp(yn2, "yn (ma tran) =");
53
54 // 7.4 nang luong cua x(n) va y(n)
55 Ex = energy(xn);
56 Ey = energy(yn1);
57 disp(Ex, "Ex =");
58 disp(Ey, "Ey =");
59
60 // Truc thoi gian n = 0..N-1
61 n = 0:length(xn) - 1;
62
63 // gioi han hien thi truc hoanh va truc tung chung
64 n_min = -1;
65 n_max = length(xn);
66 y_min = min([xn, yn1, 0]) - 1;
67 y_max = max([xn, yn1, 0]) + 1;
68
69 clf();
70
71 // x(n)
72 subplot(3, 1, 1);
73 ax1 = gca();
74 ax1.x_location = "origin";
75 ax1.box = "off";
76 plot2d3(n, xn, 2);
77 plot(n, xn, 'bo');
78 // dat gioi han truc sau khi ve de Scilab khong tu co gian lai
79 ax1.data_bounds = [n_min, y_min; n_max, y_max];
80 ax1.tight_limits = "on";
81 ax1.x_ticks = tlist(["ticks", "locations", "labels"], n_min:n_max, string(
    ↪ n_min:n_max));
82 xgrid(color('gray'));
83 title("$\text{Periodic signal } x(n), N = " + string(length(xn)) + "$", "
    ↪ fontsize", 3);
84 ylabel("$\text{Amplitude}$", "fontsize", 3);
85
86 // y(n) = x(n) (N) h(n)
87 subplot(3, 1, 2);
88 ax2 = gca();
89 ax2.x_location = "origin";
90 ax2.box = "off";
```

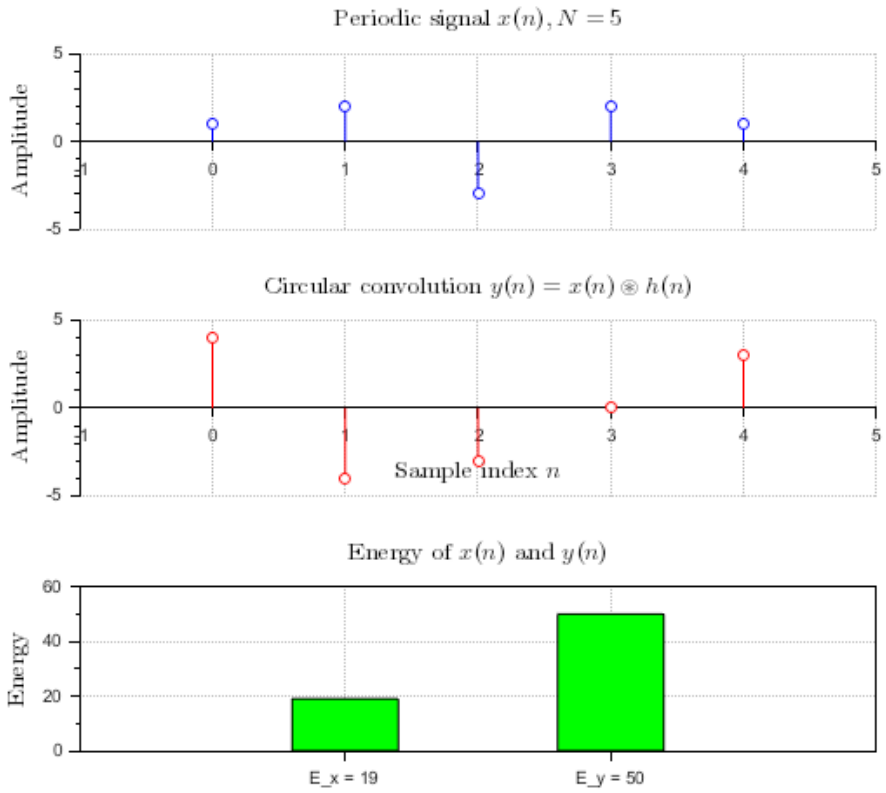
```
plot2d3(n, yn1, 5);
plot(n, yn1, 'ro');
ax2.data_bounds = [n_min, y_min; n_max, y_max];
ax2.tight_limits = "on";
ax2.x_ticks = tlist(["ticks", "locations", "labels"], n_min:n_max, string(
    ↪ n_min:n_max));
xgrid(color('gray'));
title("$\text{Circular convolution } y(n) = x(n) \circledast h(n)$", "
    ↪ fontsize", 3);
xlabel("$\text{Sample index } n$", "fontsize", 3);
ylabel("$\text{Amplitude}$", "fontsize", 3);

// nang luong E_x va E_y
subplot(3, 1, 3);
bar([1, 2], [Ex, Ey], 0.4, 'green');
ax3 = gca();
ax3.data_bounds = [0, 0; 3, max([Ex, Ey]) + 5];
ax3.x_ticks = tlist(["ticks", "locations", "labels"], [1, 2], ["E_x = " +
    ↪ string(Ex), "E_y = " + string(Ey)]);
xgrid(color('gray'));
title("$\text{Energy of } x(n) \text{ and } y(n)$", "fontsize", 3);
ylabel("$\text{Energy}$", "fontsize", 3);
```

3. Kết quả mô phỏng và nhận xét:

Scilab 2026.1.0 Console

```
--> exec('Ex_4.7.sci', -1)
   4.   -4.   -3.    0.    3.
  "yn (gap va dich) ="
   4.   -4.   -3.    0.    3.
  "yn (ma tran) ="
   19.
  "Ex ="
   50.
  "Ey ="
```



Hình 6: Tín hiệu tuần hoàn $x(n)$, tích chập vòng $y(n)$ và năng lượng của hai tín hiệu

Cả hai phương pháp đều cho $y(n) = [4, -4, -3, 0, 3]$, khớp với bảng tính tay. Khác với Bài tập 6, ở đây $y(n)$ có cùng chiều dài $N = 5$ với $x(n)$, vì các mẫu lẽ ra nằm ngoài đoạn $[0, 4]$ của tích chập tuyến tính đã quay vòng và cộng chồng lên đầu chu kỳ (hiện tượng *aliasing* trong miền thời gian). Do đó tích chập vòng chỉ trùng với tích chập tuyến tính khi $N \geq L_x + L_h - 1$; muốn dùng FFT để tính tích chập tuyến tính thì phải bù 0 cho cả hai tín hiệu đủ chiều dài này. Năng lượng $E_y = 50$, khác với E_y của tích chập tuyến tính ở Bài tập 6 vì đáp ứng xung và cách cộng chồng đều khác.

5 Bàn Luận

Các kết quả tính tay biến đổi Z ở Bài 1–3 cho thấy rõ vai trò của ROC: tín hiệu hữu hạn hội tụ gần như trên toàn mặt phẳng, tín hiệu nhân quả có ROC nằm ngoài cực lớn nhất, còn tín hiệu hai phía có ROC là vành khuyên giữa các cực. Ở phần Scilab, cả hai phương pháp tích chập (gập–dịch và ma trận) đều cho kết quả trùng nhau và trùng với hàm `conv`. Qua Bài 7 em cũng hiểu vì sao tích chập vòng khác tích chập tuyến tính: các mẫu bị quay vòng và cộng chồng. Với dữ liệu thật, Scilab đọc và phân tích được phổ của tiếng mèo grừ, còn các phép cân bằng histogram, làm mờ và watermark trên ảnh đều cho kết quả đúng như mong đợi. Khó khăn em gặp là Scilab không đọc được file `.mp3` và ảnh khi chưa cài toolbox, nên em phải chuyển âm thanh sang `.wav` và tự viết hàm đọc ảnh PGM để script chạy được trên mọi máy.

6 Tài Liệu Tham Khảo

Đường link GitHub chứa mã nguồn thực hành source code SciLab: https://github.com/qanetta/DigitalSignalProcessing-Lab.git